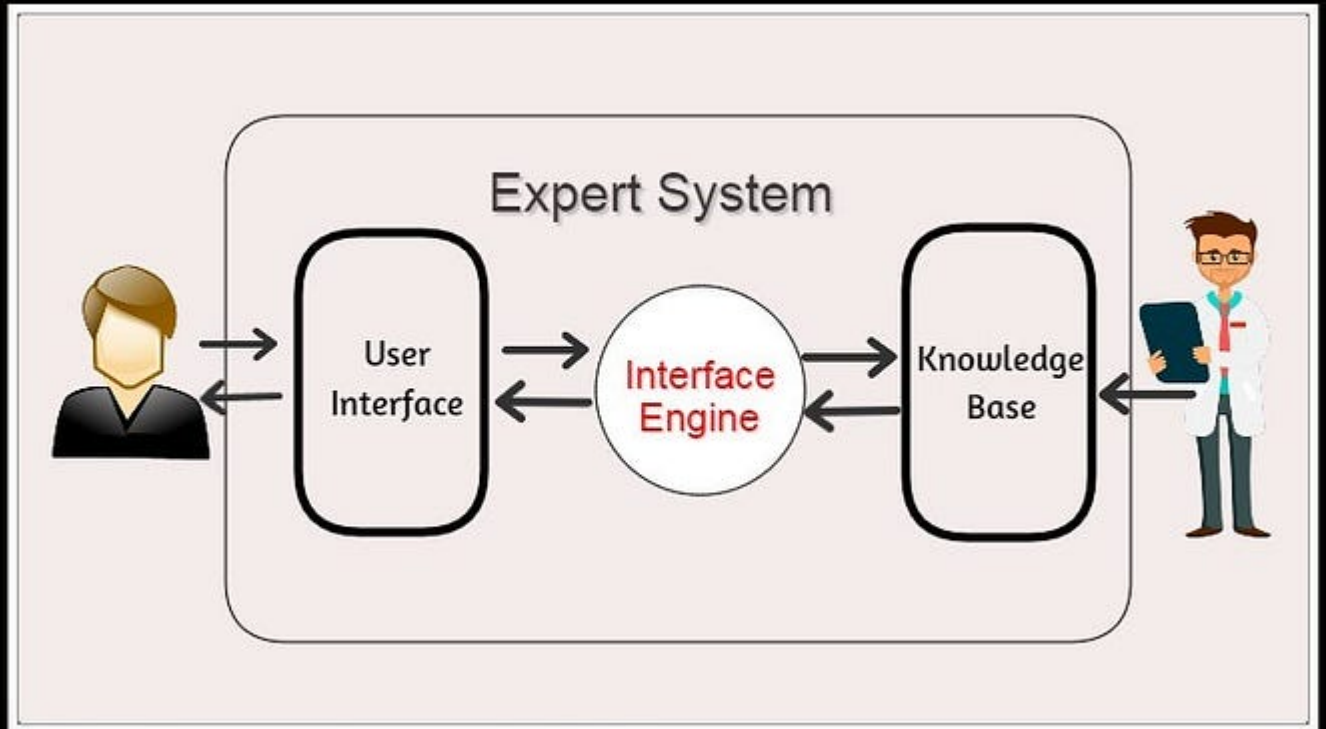


Expert System



There are 3 main components in an Expert System.

1. **User Interface** – This component helps to non-expert users to interact with the expert system and find a solution to the problem. An interface for an expert system is more complex than a interface for other systems or software solutions. Because there should be possibilities such as providing answers for asked questions, explanations, alternative solutions, capturing missing information in an expert system. In addition, Expert Systems interfaces can also handle the interaction through the natural language(voice and text). Chatbot is a good example for natural language interface for all kind of software. Today, many Expert Systems have web based interfaces. Because web interfaces can support many media such as graphic, images, voice etc.
2. **Knowledge Base** – This is a container of Expert Systems. It represents facts and rules. It consists of knowledge in a particular domain as well as rules to solve a problem, procedures and intrinsic data relevant to the domain. The knowledge may be stored in databases, web servers, file servers, or cloud. The Knowledge Base should be structured depending on requirements such as dynamic information and static information.
3. **Inference Engine** – This is a distinct feature on Expert Systems. All problem solving strategies or features of Expert Systems are built into Inference Engine. Basic Artificial Intelligence techniques such as search algorithms, forward chaining, backward chaining are considered as inference techniques. Inference Engine makes the Expert Systems different from classical database system.

Before going to discuss the topics.. Read these stories..

→ Imagine you're a doctor, and you've been practicing medicine for years. You have extensive knowledge about diseases, symptoms, and treatments. However, sometimes, there's a rare condition you haven't seen in years. How do you approach it? What if you could have the wisdom of **thousands** of doctors and medical experts available at your fingertips, guiding you through the diagnostic process?

→ Imagine a small town with only one experienced mechanic, **Tom**. Tom knows every car in town inside and out. One day, a new car model arrives, and no one knows how to fix it. Rather than waiting for Tom to learn this new model over time, the town creates an **Expert System** that mimics Tom's knowledge. Now, anyone can access this system to diagnose and fix the car, just as Tom would.

This is the essence of Expert Systems. They allow us to **capture human expertise** and make it available to others instantly, in a way that's fast and reliable. It's like having an expert on-demand, 24/7.

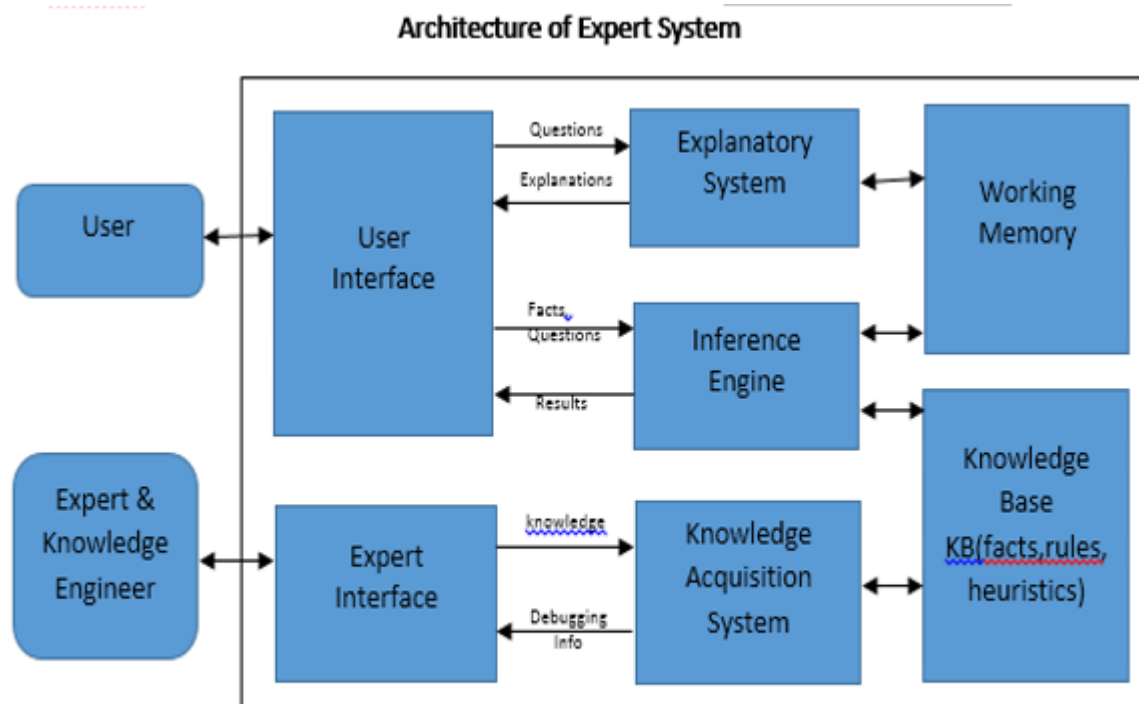


This is exactly what an **Expert System** can do.

Expert systems solve problems that are normally solved by human “experts”. To solve expert-level problems, expert systems need access to a substantial domain knowledge base, which must be built as efficiently as possible.

- An Expert System is a computer program designed to simulate the decision-making ability of a human expert. It uses **knowledge** and **inference** to solve specific problems within a given domain.
- Definition: An Expert System uses a **knowledge base** filled with expert knowledge, and an **inference engine** to make decisions or recommendations by analyzing input data.

Architecture of an Expert System



The Architecture of an Expert System (ES) consists of the following major components:

- ⊠ **Knowledge Base (KB):** repository of special heuristics or rules that direct the use of knowledge, facts (productions). It contains the knowledge necessary for understanding, formulating, & problem solving.
- ⊠ **Working Memory(Blackboard):** if forward chaining used

It describes the current problem & record intermediate results

Records Intermediate Hypothesis & Decisions: 1. Plan, 2. Agenda, 3. Solution

- ⌘ Inference Engine: the deduction system used to infer results from user input & KB

It is the brain of the ES, the control structure(rule interpreter)

It provides methodology for reasoning

- ⌘ Explanation Subsystem (Justifier): Traces responsibility & explains the ES behaviour by interactively answering question: Why?, How?, What?, Where?, When?, Who?
- ⌘ User Interface: interfaces with user through Natural Language Processing (NLP), or menus & graphics. Acts as Language Processor for friendly, problem-oriented communication

Components of Expert Systems:

- Knowledge Base
 - The heart of any Expert System, the **knowledge base** is a collection of facts, rules, and information from experts in the specific field. This is what the system uses to make decisions or provide solutions.



- **Real-Life Example:** In a medical expert system, the knowledge base would contain detailed information on symptoms, diseases, medications, treatments, etc.

2. Inference Engine

- This is the "**brain**" of the Expert System. It applies logical reasoning to the knowledge base and works out conclusions or solutions based on the input it receives from the user.
- **Real-Life Example:** Imagine you're using a tax advice expert system. You input details about your income and deductions. The inference engine processes the knowledge base to calculate your tax liability.

3. User Interface

- The **User Interface (UI)** allows people to interact with the Expert System. It helps users input their data and get results in a way that is easy to understand and use.
- **Real-Life Example:** When you use an online help chatbot for customer service, the UI allows you to type questions and get answers.

4. Explanation Facility

- This characteristic enables the system to explain how it arrived at a particular conclusion or decision. It adds transparency and helps users trust the system's output.
- **Real-Life Example:** A legal expert system may explain how it reached a conclusion about the possible outcomes of a case based on the laws and previous rulings it knows.

5. Knowledge Acquisition

- Knowledge Acquisition refers to the process of **gathering** and adding new knowledge to the Expert System, typically from human experts.
- **Real-Life Example:** If a car repair expert system needs to learn how to fix a new car model, it will acquire knowledge from an experienced mechanic.

History:

1. The Early Beginnings (1950s-1970s)



In the **1950s and 1960s**, computer scientists started to explore ways to simulate human intelligence. However, back then, computers were primarily used for mathematical calculations and data processing, not for simulating human expertise.

- **Key Event:** Early AI systems focused on symbolic reasoning and logic but did not yet have the ability to mimic specialized human expertise.

2. Birth of Expert Systems (1970s)

In the **1970s**, Expert Systems emerged as a major breakthrough. The goal was to create a system that could simulate the decision-making of a human expert. This period marked the **formal birth** of Expert Systems.

- **Key Event:** The first real Expert System, **DENDRAL**, was developed at Stanford University in 1965. DENDRAL helped chemists identify molecular structures based on mass spectrometry data.
 - **Real-Life Example:** Think of how a modern **AI medical diagnosis tool** helps doctors with diagnosing rare diseases — this concept started with systems like DENDRAL.
- **Key Event:** Another key Expert System, **MYCIN** (developed in the early 1970s), was designed to diagnose bacterial infections and recommend treatments. It used **rule-based reasoning** to mimic how expert doctors made decisions.

3. Growth and Popularity (1980s)

In the **1980s**, Expert Systems saw **explosive growth**. Companies and industries began realizing how useful these systems could be in solving complex problems. **Rule-based Expert Systems** became widespread, especially in fields like medicine, engineering, and finance.

- **Key Event:** **XCON**, a popular Expert System developed by DEC (Digital Equipment Corporation), helped configure computer systems by using expert knowledge to assemble computer parts.
 - **Real-Life Example:** Think of how modern configuration systems in e-commerce websites help customers select the right product based on their needs.



4. The Decline and Challenges (1990s)

In the **1990s**, Expert Systems began facing limitations:

- They were **expensive** to maintain because knowledge needed to be continuously updated by human experts.
- Systems became **rigid** and couldn't handle uncertainty or new information without major updates.

However, they still found use in specialized areas where they could deliver high-value solutions.

5. Modern-Day Expert Systems (2000s-Present)

Today, Expert Systems have evolved with the integration of machine learning, big data, and AI. They are **more adaptive, flexible**, and can process huge amounts of unstructured data to make better predictions and decisions.

- **Key Event:** AI systems like **IBM Watson** (famous for winning **Jeopardy!**) are modern forms of Expert Systems that use AI to process and analyze vast amounts of data to provide expert-level recommendations in fields like healthcare, law, and customer service.
 - **Real-Life Example:** The AI behind **Amazon's recommendation system** acts like an expert system, guiding customers to purchase products based on their past preferences and choices

Applications:

- **Medical Diagnosis:** Expert systems assist doctors by analyzing symptoms and medical history to suggest possible diagnosis or treatment options. For example, MYCIN, an early expert system, helped identify bacterial infections and recommend antibiotics.
- **Financial Services:** In finance, expert systems are used for credit scoring, fraud detection, and investment advice. They analyze financial data and patterns to make informed decisions.



- **Technical Support:** Expert systems can troubleshoot and provide solutions for technical issues. They guide users through problem-solving steps based on pre-defined rules and knowledge.
- **Manufacturing:** In manufacturing, expert systems help optimize production processes, perform quality control, and manage inventory by analyzing data and making recommendations.

Design Domain : Camera lens design, automobile design.

CHARACTERISTICS OF AN EXPERT SYSTEM:

1. **High level Performance:** The system must be capable of responding at a level of competency equal to or better than an expert system in the field. The quality of the advice given by the system should be in a high level integrity and for which the performance ratio should be also very high.
2. **Domain Specificity:** Expert systems are typically very domain specific. For ex., a diagnostic expert system for troubleshooting computers must actually perform all the necessary data manipulation as a human expert would. The developer of such a system must limit his or her scope of the system to just what is needed to solve the target problem. Special tools or programming languages are often needed to accomplish the specific objectives of the system.
3. **Good Reliability:** The expert system must be as reliable as a human expert.
4. **Understandable:** The system should be understandable i.e. be able to explain the steps of reasoning while executing. The expert system should have an explanation capability similar to the reasoning ability of human experts.



5. Adequate Response time: The system should be designed in such a way that it is able to perform within a small amount of time, comparable to or better than the time taken by a human expert to reach at a decision point. An expert system that takes a year to reach a decision compared to a human expert's time of one hour would not be useful.
6. Use symbolic representations: Expert system use symbolic representations for knowledge (rules, networks or frames) and perform their inference through symbolic computations that closely resemble manipulations of natural language.
7. Linked with Metaknowledge: Expert systems often reason with metaknowledge i.e. they reason with knowledge about themselves and their own knowledge limits and capabilities. The use of metaknowledge is quite interactive and simple for various data representations.
8. Expertise knowledge: Real experts not only produce good solutions but also find them quickly. So, an expert system must be skillful in applying its knowledge to produce solutions both efficiently and effectively by using the intelligence human experts.
9. Justified Reasoning: This allows the users to ask the expert system to justify the solution or advice provided by it. Normally, expert systems justify their answers or advice by explaining their reasoning. If a system is a rule based system, it provides to the user all the rules and facts it has used to achieve its answer.
10. Explaining capability: Expert systems are capable of explaining how a particular conclusion was reached and why requested information is needed during a consultation. This is very important as it gives the user a chance to access and understand the system's reasoning ability, thereby improving the user's confidence in the system.



11. Special Programming Languages: Expert systems are typically written in special programming languages. The use of languages like LISP and PROLOG in the development of an expert system simplifies the coding process. The major advantage of these languages, as compared to conventional programming languages is the simplicity of the addition, elimination or substitution of new rules and memory management capabilities. Some of the distinguishing characteristics of programming languages needed for expert system work are as follows:

Efficient mix of integer and real variables.

Good memory management procedures.

Extensive data manipulation routines.

Incremental compilation.

Tagged memory architecture.

Efficient search procedures.

Optimization of the systems environment.

Rule based System:

A **Rule-Based System (RBS)** is a type of **knowledge-based system** that uses a set of **rules** to represent expert knowledge. The rules are usually in the form of "IF... THEN..." statements. The system applies these rules to draw conclusions and make

decisions. These systems are crucial in **expert systems** because they provide the logic that helps solve complex problems.

How Does Rule-Based System Architecture Relate to Expert Systems?

- **Core Concept:** Rule-Based Systems are a **core component** of many **Expert Systems**. They are used to encode knowledge and solve problems through logical reasoning.
- **Knowledge Representation:** In expert systems, knowledge is typically represented using **rules**. These rules describe the **relationships** between different facts, such as:
 - "IF a patient has a fever AND a cough, THEN the diagnosis could be a respiratory infection."
- **Inference Engine:** The **Inference Engine** in Expert Systems is essentially the **rule interpreter** that applies the rules from the knowledge base to deduce conclusions or provide solutions. It evaluates the rules in the knowledge base based on the input provided by the user.
 - **Example:** In a **Medical Diagnosis Expert System**, the **Inference Engine** would apply rules like the one above to suggest possible diagnoses based on the input symptoms (fever, cough, etc.).



Core Components of Rule-Based Systems

A typical Rule-Based System consists of the following key components:

1. Knowledge Base:

- The **Knowledge Base** stores all the facts, rules, and domain-specific knowledge that the system uses to reason and make decisions.
- **Example:** In a medical system, the knowledge base would contain rules like:
 - "IF the patient has a fever AND cough, THEN the diagnosis could be flu."

2. Inference Engine:

- The **Inference Engine** is the part of the system that **applies the rules** in the knowledge base to the facts provided as input, in order to draw conclusions or make decisions.
- It works by **matching** the input data with the rules and then applying the rules to infer new information.
- **Example:** Given the input data (fever, cough), the inference engine might trigger the rule that suggests a flu diagnosis.

3. Working Memory:

- The **Working Memory** temporarily holds the current set of facts (input data) that the system is processing. It stores facts that are derived during the execution of the rules.
- **Example:** In a medical system, facts like "Patient has fever" or "Patient has a cough" will be stored in working memory.

4. User Interface:

- The **User Interface (UI)** allows the user to interact with the system. It collects the input (facts or symptoms) from the user and displays the results or recommendations made by the system.
 - **Example:** The user might input symptoms like fever and cough through a text box or form.
-



2. How Rule-Based Systems Work

Rule-Based Systems work in the following sequence:

1. **Input Data:** The user provides input data (facts or symptoms).
 - **Example:** "Patient has a fever" and "Patient has a cough."
2. **Rule Matching:** The system's **Inference Engine** checks all the rules in the Knowledge Base to find those that match the input data (facts).
 - **Example:** It finds the rule: "IF the patient has fever AND cough, THEN the diagnosis could be flu."
3. **Inference:** Once a matching rule is found, the Inference Engine applies it to deduce new facts or conclusions.
 - **Example:** The system might infer that the patient has the flu.
4. **New Facts/Conclusions:** The new facts or conclusions are stored in **Working Memory** and may trigger further rules or actions.
 - **Example:** The system might trigger a rule that suggests flu treatment based on the diagnosis.
5. **Output:** The system then presents the results, advice, or diagnosis to the user.
 - **Example:** The system might display a message saying, "The patient is diagnosed with the flu. Recommended treatment: Rest and fluids."

3. Types of Reasoning in Rule-Based Systems

Rule-Based Systems typically use two types of reasoning to derive conclusions:

1. **Forward Chaining (Data-Driven):**
 - **Forward Chaining** starts with known facts and applies rules to deduce new facts until a goal or conclusion is reached.
 - It's **data-driven** because the system starts with the available data and applies rules to move forward.
 - **Example:** Starting with facts like "fever" and "cough", the system applies rules to infer that the patient has the flu.
2. **Backward Chaining (Goal-Driven):**
 - **Backward Chaining** starts with a goal or hypothesis and works backward to see if there's data that supports it.



- It's **goal-driven** because the system tries to prove that a conclusion or hypothesis is true by finding supporting evidence.
- **Example:** The system starts with the goal "Does the patient have the flu?" and works backward to find symptoms or facts that confirm it.

4. Example of Rule-Based System: Medical Diagnosis

Let's take the example of a **Medical Diagnosis System** that uses Rule-Based Architecture:

1. **Knowledge Base:** Contains rules like:
 - "IF the patient has fever AND cough, THEN the diagnosis could be flu."
 - "IF the patient has shortness of breath AND chest pain, THEN the diagnosis could be heart attack."
2. **Inference Engine:** Takes input (e.g., fever, cough) and checks the knowledge base to apply the relevant rules. It triggers the rule and concludes that the patient might have the flu.
3. **Working Memory:** Stores the facts (fever, cough, etc.) during the process.
4. **Output:** The system suggests the diagnosis (e.g., flu) and may recommend treatment or further tests.

5. Advantages of Rule-Based Systems

1. **Transparency:** The reasoning process is clear because the rules are explicit. Users can see how conclusions are drawn.
2. **Flexibility:** Rules can be easily modified or added without changing the entire system.
3. **Simple to Understand:** Rules are usually written in simple, human-readable language, making them easy to interpret.

6. Limitations of Rule-Based Systems

1. **Knowledge Acquisition:** Creating and maintaining the knowledge base can be time-consuming and requires constant updates from experts.
2. **Lack of Flexibility in Complex Situations:** If the situation doesn't fit a predefined rule, the system might fail or provide incorrect results.

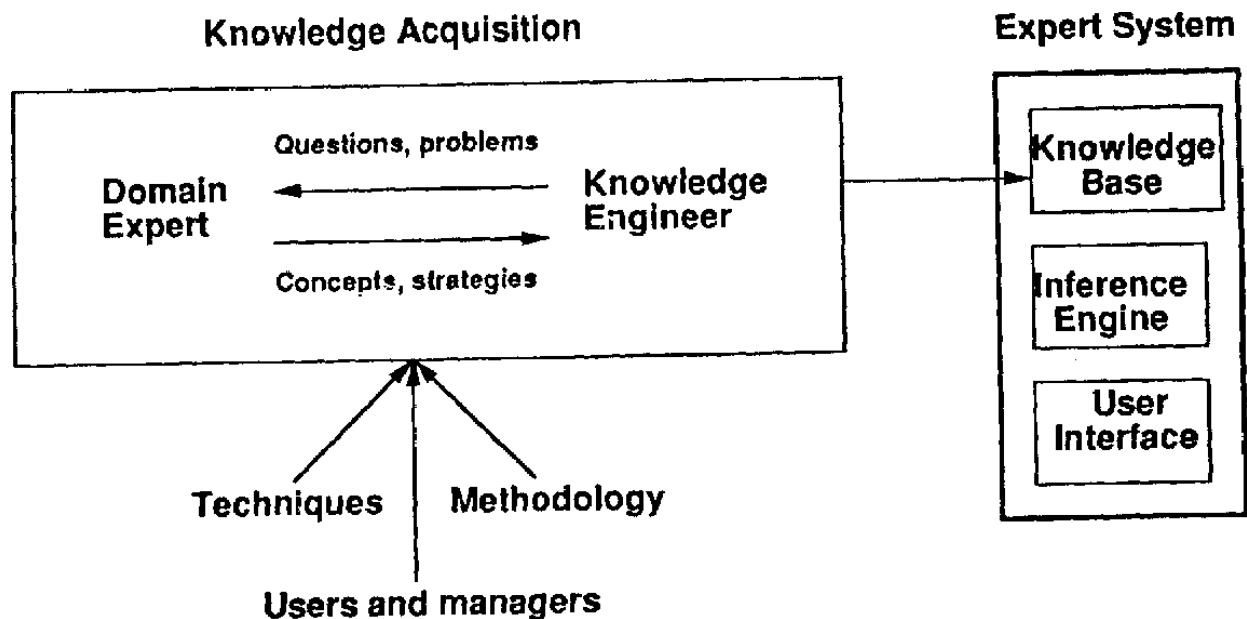


3. **Performance Issues:** For large knowledge bases, the system may become slow as it checks many rules for every new input.

For more information refer : [Expert Systems in AI - GeeksforGeeks](#)

[Expert Systems: What it is and Why it Matters?](#)

Knowledge Acquisition:



How are expert systems built? Typically, a knowledge engineer interviews a domain expert to elucidate expert knowledge, which is translated into rules. After the initial system is built, it must be iteratively refined until it approximates expert-level performance. This process is expensive and time-consuming, so it is worthwhile to look for more automatic ways of constructing expert knowledge space.

Knowledge acquisition is the gathering or collecting knowledge from various sources. It is the process of adding new knowledge to a knowledge base and refining or improving knowledge that was previously acquired. Acquisition is the process of expanding the capabilities of a system or improving its performance at some specified task. So it is the goal oriented creation and refinement of knowledge.

References : [Artificial Intelligence: Knowledge Acquisition | PPT](#)

Types of Learning:

Knowledge acquisition in expert systems refers to how an intelligent system gains knowledge, improves its decision-making, and refines its reasoning over time. There are several learning methods, each playing a crucial role in developing expert systems.

A. Rote Learning (Memorization-based Learning)

- Rote learning is the simplest form of knowledge acquisition.
- The system directly stores information without analyzing or modifying it.
- There is no reasoning involved—just raw memorization.
- **Example:** A chatbot storing predefined responses to FAQs. It does not understand the meaning but recalls stored replies when needed.

B. Inductive Learning (Pattern Recognition)

- In inductive learning, the system generalizes knowledge from past experiences or examples.
- It identifies patterns in data and forms general rules.
- **Example:** A spam email filter learning patterns from flagged emails and applying rules to new ones.

Process of Inductive Learning:

1. **Data Collection** – The system gathers a dataset of examples (e.g., emails labeled as spam or not spam).
2. **Pattern Identification** – It identifies common features in spam emails (e.g., frequent use of "free," "win," and "urgent").
3. **Rule Formation** – It generates a general rule (e.g., "If an email contains more than 3 promotional words, classify it as spam").

C. Deductive Learning (Rule-Based Learning)

- Deductive learning applies **existing** rules to reach conclusions.



- Unlike inductive learning, which discovers patterns, deductive learning follows logical reasoning based on predefined facts.
- **Example:**
 - Rule: "All fruits contain vitamins."
 - Fact: "An apple is a fruit."
 - Conclusion: "An apple contains vitamins."

D. Analogical Learning (Learning by Comparison)

- The system compares a new situation to past experiences to derive conclusions.
- **Example:** A doctor diagnosing a rare disease based on similarities with previously seen cases.

How It Works:

1. The system retrieves a similar case from memory.
2. It compares features of the new and old case.
3. If similarities are strong, it applies the previous knowledge to the new case.

E. Reinforcement Learning (Trial & Error Learning)

- The system learns by interacting with an environment and receiving feedback in the form of rewards or penalties.
- **Example:** A self-driving car adjusting its braking and acceleration based on real-time performance.

Process:

1. The system **performs an action**.
2. It receives **a reward or punishment** based on the outcome.
3. The system **adjusts its strategy** to maximize rewards.



General Learning Model:

The process of knowledge acquisition in expert systems follows a structured approach. A **General Learning Model** provides a systematic way of learning.

Steps in a General Learning Model:

1. Data Collection

- The system gathers information from databases, experts, sensors, or previous cases.
- **Example:** A medical AI system collecting patient symptoms and test results.

2. Preprocessing

- Raw data is cleaned and organized to remove inconsistencies.
- **Example:** Removing duplicate or irrelevant patient records.

3. Feature Selection

- The system identifies the most relevant attributes for decision-making.
- **Example:** A spam filter selecting words like "free" and "urgent" as key features.

4. Model Training

- The system learns by processing past data and generating rules or patterns.
- **Example:** A fraud detection model trained on past fraudulent transactions.

5. Testing & Evaluation



- The system is tested on new data to evaluate its accuracy.
- **Example:** Testing an AI-powered medical diagnostic system on new patient cases.

6. Knowledge Representation

- The acquired knowledge is stored in a structured way (rules, decision trees, etc.).
- **Example:** A self-driving car storing learned road navigation rules.

Performance Measures:

- 1) Accuracy
- 2) Precision and Recall
- 3) Speed of Learning
- 4) Robustness
- 5) Adaptability

